

Introducción teórico-práctica al Aprendizaje por Refuerzo

Clase 4— Dynamic Programming

Profesor: Matias Bilkis



UBA
Universidad de Buenos Aires



¿Cómo venimos?



Russian mathematician
Andrey Markov



¿Qué vimos la clase pasada?

Retorno descontado $\rightarrow G_t = R_{t+1} + \gamma G_{t+1}$

Expandir $\mathbb{E}_\pi$ por un time-step $\rightarrow$ **Bellman eqs.**

$$v_\pi(s) = \sum_{r,a} r p(r|s,a) \pi(a|s) + \gamma \sum_{a,s'} \pi(a|s) p(s'|s,a) v_\pi(s')$$

¿Qué vimos la clase pasada?

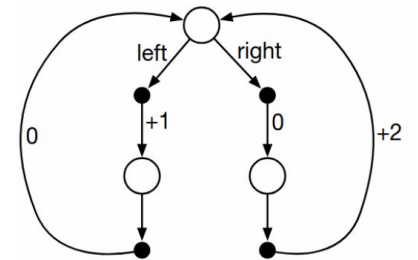
Retorno descontado $\rightarrow G_t = R_{t+1} + \gamma G_{t+1}$

Expandir $\mathbb{E}_\pi$ por un time-step $\rightarrow$ **Bellman eqs.**

$$v_\pi(s) = \sum_{r,a} r p(r|s,a) \pi(a|s) + \gamma \sum_{a,s'} \pi(a|s) p(s'|s,a) v_\pi(s')$$

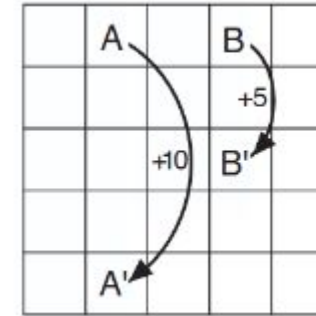
¿Quedó claro? ¿Hicieron la tarea? ¿Se entendió?

¿A alguien le interesa? $\rightarrow$



¿Qué vimos la clase pasada?

Si al Gólem le das las value-functions...



3.3	8.8	4.4	5.3	1.5
1.5	3.0	2.3	1.9	0.5
0.1	0.7	0.7	0.4	-0.4
-1.0	-0.4	-0.4	-0.6	-1.2
-1.9	-1.3	-1.2	-1.4	-2.0



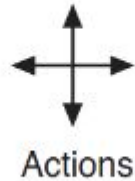
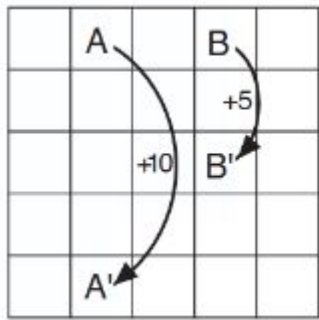
El problema del Gólem radica en cómo construye su policy “óptima” si solamente tiene los value-functions

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a] = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s, A_t = a\right]$$

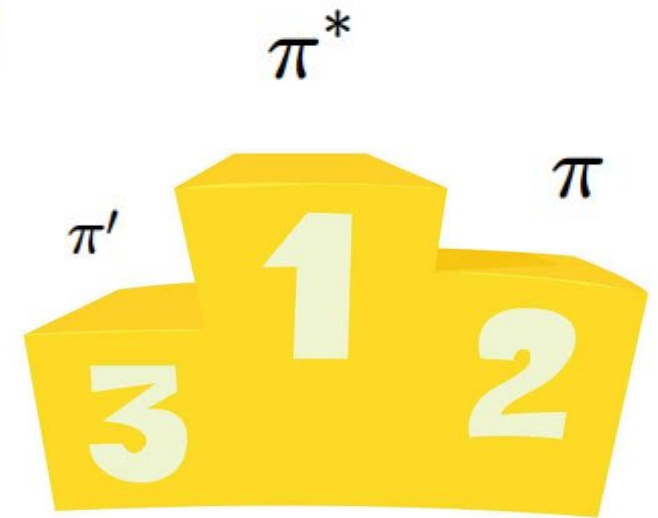
¿Qué vimos la clase pasada?

El problema del RL radica en encontrar **la mejor policy**

$$\pi \geq \pi' \iff v_{\pi}(s) \geq v_{\pi'}(s) \quad \forall s \in \mathcal{S}$$



3.3	8.8	4.4	5.3	1.5
1.5	3.0	2.3	1.9	0.5
0.1	0.7	0.7	0.4	-0.4
-1.0	-0.4	-0.4	-0.6	-1.2
-1.9	-1.3	-1.2	-1.4	-2.0



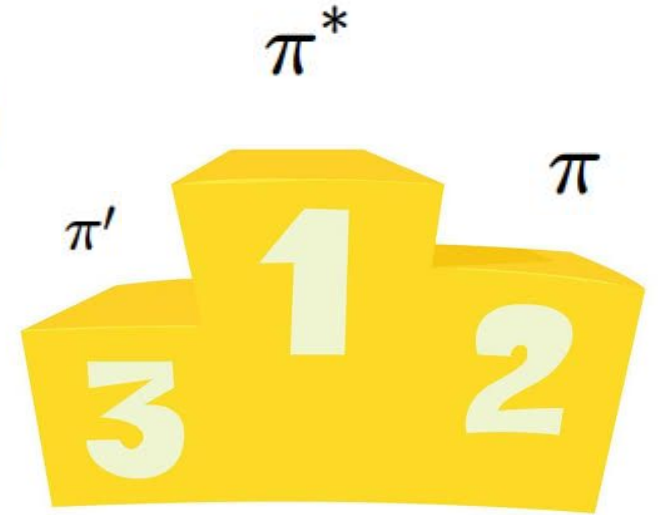
$$v_*(s) := \max_{\pi} v_{\pi}(s)$$

¿Qué vimos la clase pasada?

Las políticas óptimas también son políticas!
Optimal Bellman equations!

$$v_*(s) = \max_a \left(\sum_{r,a} r p(r|s, a) + \gamma \sum_{s'} p(s'|s, a) v_*(s') \right)$$

$$Q_*(s, a) = \sum_r r p(r|s, a) + \gamma \max_{a'} \sum_{s'} p(s'|s, a) Q_{\pi}(s', a')$$



Una buena “*pregunta de parcial*”:

Si nos dan la $Q_*(s, a)$... **¿cómo construimos π^* ?**

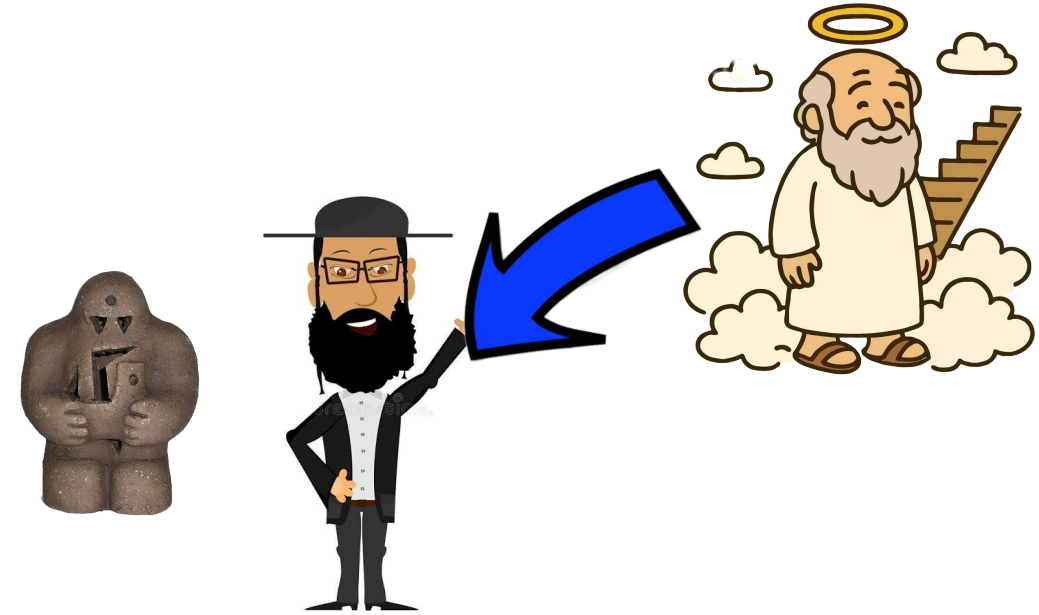
¿Qué nos interesa?



¿Se acuerdan cuando éramos jóvenes, Clase 1?

El rabí le explicaba el universo //
esto es mi pie; esto el tuyo, esto la sogá //

y logró, al cabo de años, que el perverso //
barriera bien o mal la sinagoga



¿De dónde sacamos $p(s'r|sa)$?

Una idea: hacer un modelito “físico” y reconstruir probs.



Pero incluso reconstruyendo las transiciones $p(s'|sa)$ de un “emulador”,
cómo se traduce en “ $p(r|s,a)$ ”? **Difícil**

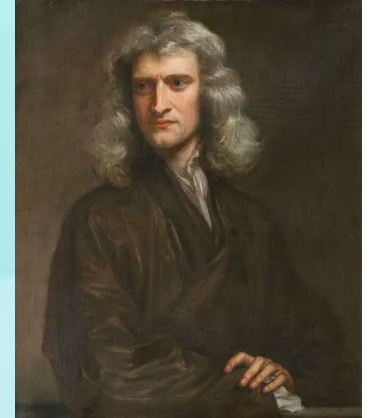
¿Cuál es mi punto acá?

Si barre OK $\rightarrow$ premio
Si NO barre premio = 0

Noción de prueba & error



Leyes para
 $s \rightarrow s', r$
(ecuaciones, y
eventualmente
reconstruyo)



$p(s'r|s,a)$

*¿Pero cómo haría
el Rabino para
traducir sus
intenciones en
ecuaciones?*

[Inverse Reinforcement Learning]

¿Qué pasa con Atari?



- El emulador tiene sus propias leyes dinámicas
- Podríamos jugar a “decodificar” $p(s'r|s,a)$:
 - * juntando estadística y reconstruyendo
 - * usando algún otro método más inteligente
 - * Entender el rol de r difícil, pues señal/regla externa!

¿Qué pasa con Atari?



- La consola tiene 128 bytes = 1024 bits de memoria.
- El value function depende de otros $2^{2014} - 1$ $v(s)$!
- No solo no podemos guardar $v(s)$, tampoco iterar...

$$v_{\pi}(s) = \sum_{r,a} r p(r|s,a) \pi(a|s) + \gamma \sum_{a,s'} \pi(a|s) p(s'|s,a) v_{\pi}(s')$$

[The Arcade Learning Environment](#)

Moraleja



- Incluso el juego de “ser dios” que nos proponemos, es complicado!
- Lo que vamos a hacer ahora va a ser asumir que conocemos las dinámicas $p(s'r|sa)$ y estudiar algunos algoritmos para resolver el “problema de RL”.

Plan de la clase

- Policy prediction
- Policy improvement
- Policy iteration
- Value iteration
- Generalized policy iteration
- Gymnasium (farama)
- Ejemplitos

Policy evaluation

Contractividad de la Eq. Bellman

- Pensemos en los elementos del RL:
 - ** Policy $\pi(a|s)$
 - ** Environment dynamics $p(s'r|sa)$
 - ** Factor de descuento γ
- Distintos elementos (o combinaciones de) dan diferentes value functions $v(s)$. **Obvio, ¿no?**



$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s] = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s\right]$$

Contractividad de la Eq. Bellman

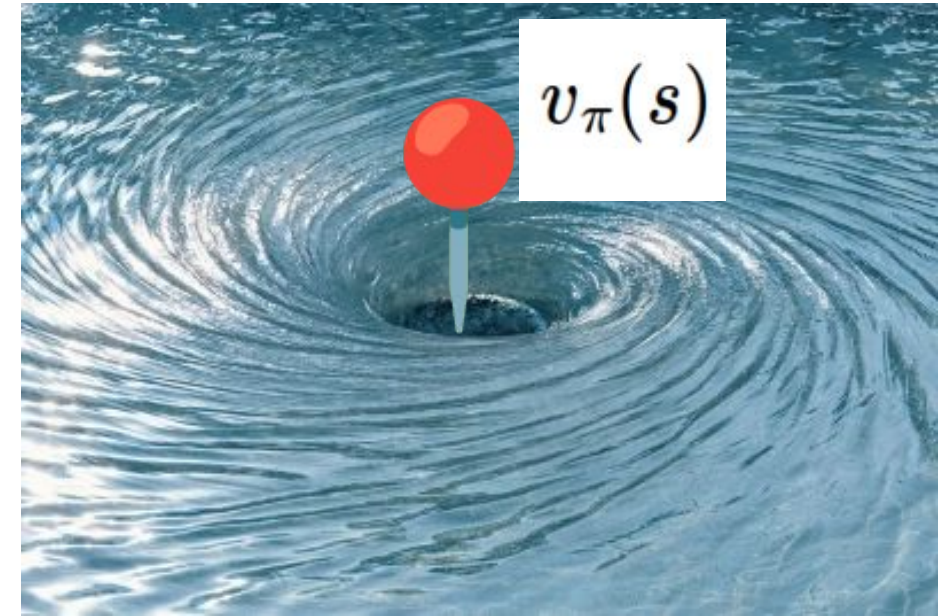
- Pensemos en los elementos del RL:
 - ** Policy $\pi(a|s)$
 - ** Environment dynamics $p(s'r|sa)$
 - ** Factor de descuento γ
- Para cada “configuración” del RL, vamos a tener una Ec. de Bellman diferente (*que no solo depende de la policy!*)



$$v_{\pi}(s) = \sum_{r,a} r p(r|s,a) \pi(a|s) + \gamma \sum_{a,s'} \pi(a|s) p(s'|s,a) v_{\pi}(s')$$

Contractividad de la Eq. Bellman

- Pensemos en los elementos del RL:
 - ** Policy $\pi(a|s)$
 - ** Environment dynamics $p(s'r|sa)$
 - ** Factor de descuento γ

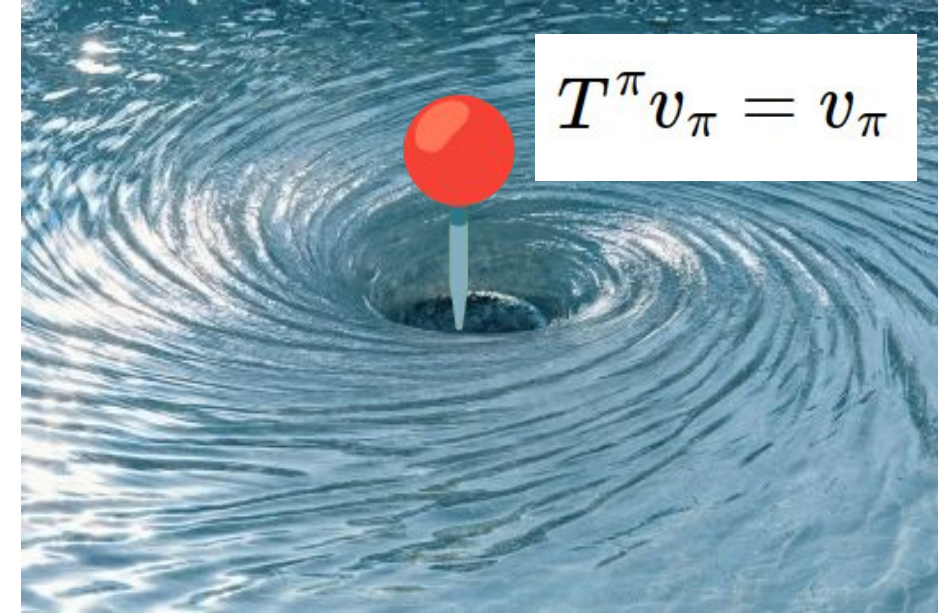


- Para cada “configuración” del RL, vamos a tener una Ec. de Bellman diferente (*que no solo depende de la policy!*)

$$v_\pi(s) = \sum_{r,a} r p(r|s, a) \pi(a|s) + \gamma \sum_{a,s'} \pi(a|s) p(s'|s, a) v_\pi(s')$$

Contractividad de la Eq. Bellman

- Si pensamos en un operador que actúa sobre funciones de los estados...
 - no necesariamente $v_{\pi}(s)$ -

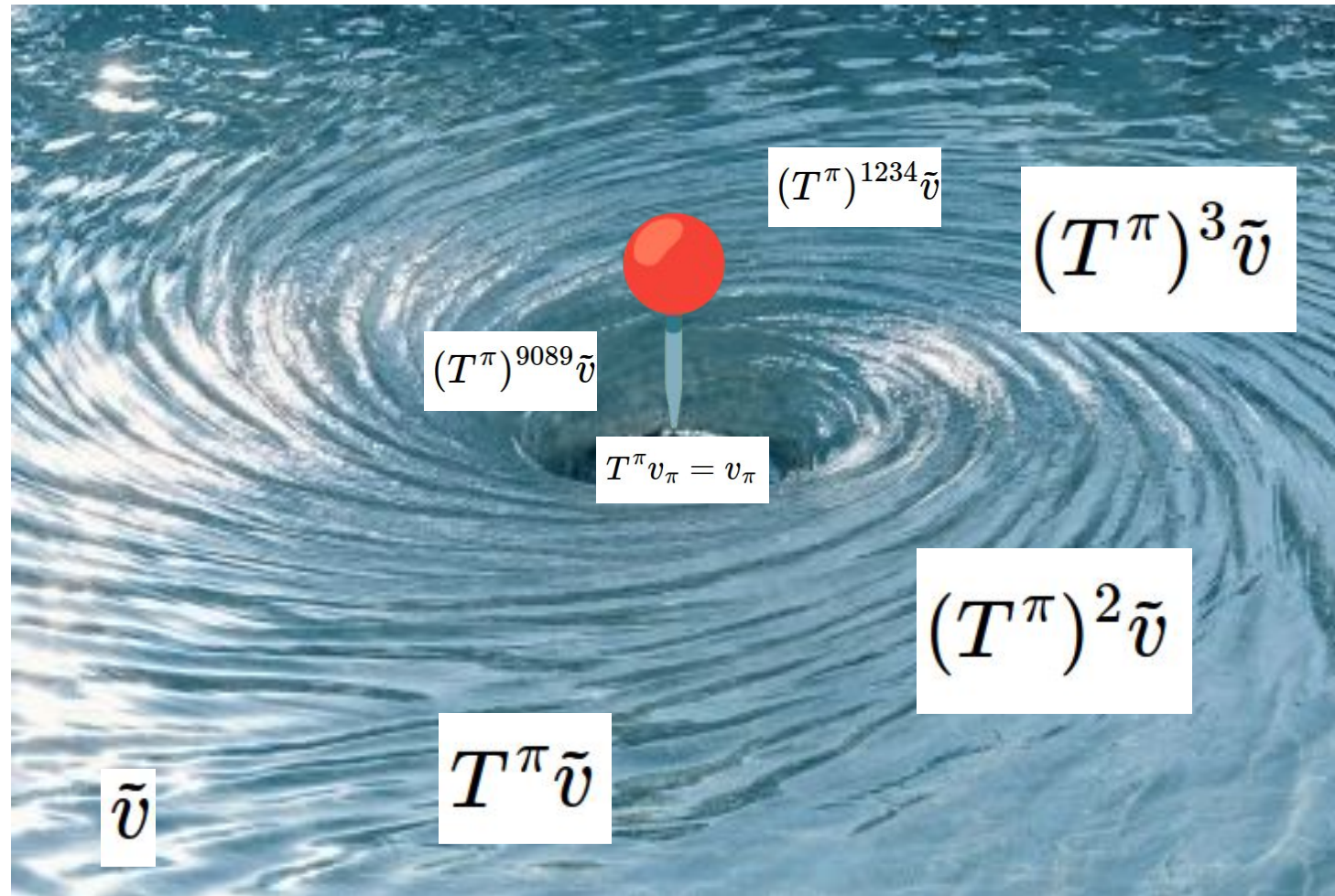


$$(T^{\pi}v)(s) := \sum_{r,a} r, p(r|s, a)\pi(a|s) + \gamma \sum_{a,s'} \pi(a|s)p(s'|s, a)v(s')$$

Y lo que dice la Ec. de Bellman es: $T^{\pi}v_{\pi} = v_{\pi}$

Contractividad de la Eq. Bellman

- ¿¿Qué pasa si aplicamos T^π a un vector genérico $\tilde{v}$??



$$T^\pi \tilde{v} = ??$$

Predicción del value function



```
Loop:
   $\Delta \leftarrow 0$ 
  Loop for each  $s \in \mathcal{S}$ :
     $v \leftarrow V(s)$ 
     $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$ 
     $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
until  $\Delta < \theta$ 
```

termination criteria
(si ya no cambia)

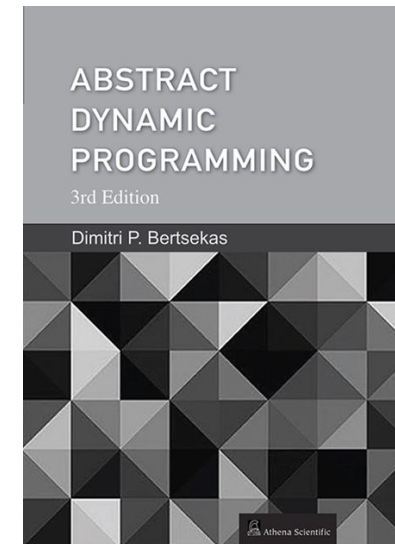
“bootstrapping”
(uso los $v(s)$ que
voy modificando)

Más detalles (sobre todo teóricos)

- Para $\gamma < 1$, $v_{\pi}(s)$ existe y es único.
- $\gamma = 1$, solo si los episodios son de horizonte finito.



[Lecture 4 curso DeepMind](#)



[Abstract Dynamic Programming](#)

Predicción del value function

```
Loop:
  Δ ← 0
  Loop for each s ∈ S:
    v ← V(s)
    V(s) ← ∑a π(a|s) ∑s',r p(s',r|s,a) [r + γV(s')]
    Δ ← max(Δ, |v - V(s)|)
until Δ < θ
```



termination criteria
(si ya no cambia)

“bootstrapping”
(uso los v(s) que
voy modificando)

TERMINAL STATE



actions

	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

TERMINAL STATE

$R_t = -1$
on all transitions

v_k for the
random policy

$k = 0$

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

$k = 3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

$k = 10$

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0

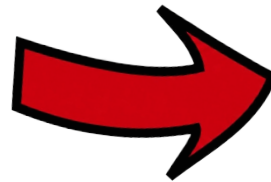
$k = \infty$

0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0

Policy improvement

Predicción del value function

- Una vez que “aprendimos” los v_π , nos gustaría mejorar!
- Para un estado s , si en lugar de seguir π nos iría mejor con otra receta? **¿Qué objeto nos puede indicar esto?**



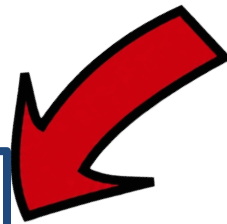
Policy improvement theorem

- Si existe una acción $\tilde{a}$ que es mejor que lo que dice $\pi(\cdot|s)$

Entonces tiene que suceder que $Q_{\pi}(s, \tilde{a}) \geq v_{\pi}(s)$

En tal caso definimos $\pi'(s) = \tilde{a}$

$\pi' \geq \pi$



Si existe una mejor acción, entonces hay mejor policy

¿Cómo *mejoramos* la policy?

Relación entre los dos objetos

- Volvamos a la recurrencia: $G_t = R_{t+1} + \gamma G_{t+1}$

$$Q_\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s, A_t = a\right]$$

- Si reemplazamos en cada uno de los términos, obtenemos

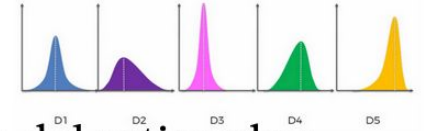
$$Q_\pi(s, a) = \sum_r r p(r|s, a) + \gamma \sum_{s'} p(s'|s, a) v_\pi(s')$$

¿¿ **Cómo sería la Ecuación de Bellman para las Q??**

46

Clase 3, slide 46
(Q en función de v)

Greedy actions



Imaginemos que estamos muy seguros del estimador...

$$A_t = \operatorname{argmax}_a \hat{q}_t(a)$$

Greedy = “codiciosa / comilona”

ε-greedy ¿Pero si queremos explorar?

$$A_t = \begin{cases} \operatorname{argmax}_a \hat{q}_t(a) & \text{con prob} = 1 - \epsilon \\ \text{random} & \text{con prob} \epsilon \end{cases}$$

19

Clase 2, slide 19
(greedy actions)

¿Cómo *mejoramos* la policy?

- Sabemos que existe una acción $\tilde{a}$ c/ mayor retorno (exp.)
- Construimos una nueva policy:

$$\pi'(s) = \max_a Q_{\pi}(s, a)$$

- En términos de $v(s)$

$$\pi'(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) \left[r + \gamma v_{\pi}(s') \right]$$

- Es bonito, porque si no mejora, entonces

$$v_{\pi'}(s) = \max_a Q_{\pi}(s, a) = \max_a \sum_{s', r} p(s', r | s, a) \left[r + \gamma v_{\pi}(s') \right]$$



Policy improvement

- Estimamos los $v(s)$ via policy evaluation
- Para cada estado, “mejoramos” según:

For each $s \in \mathcal{S}$:

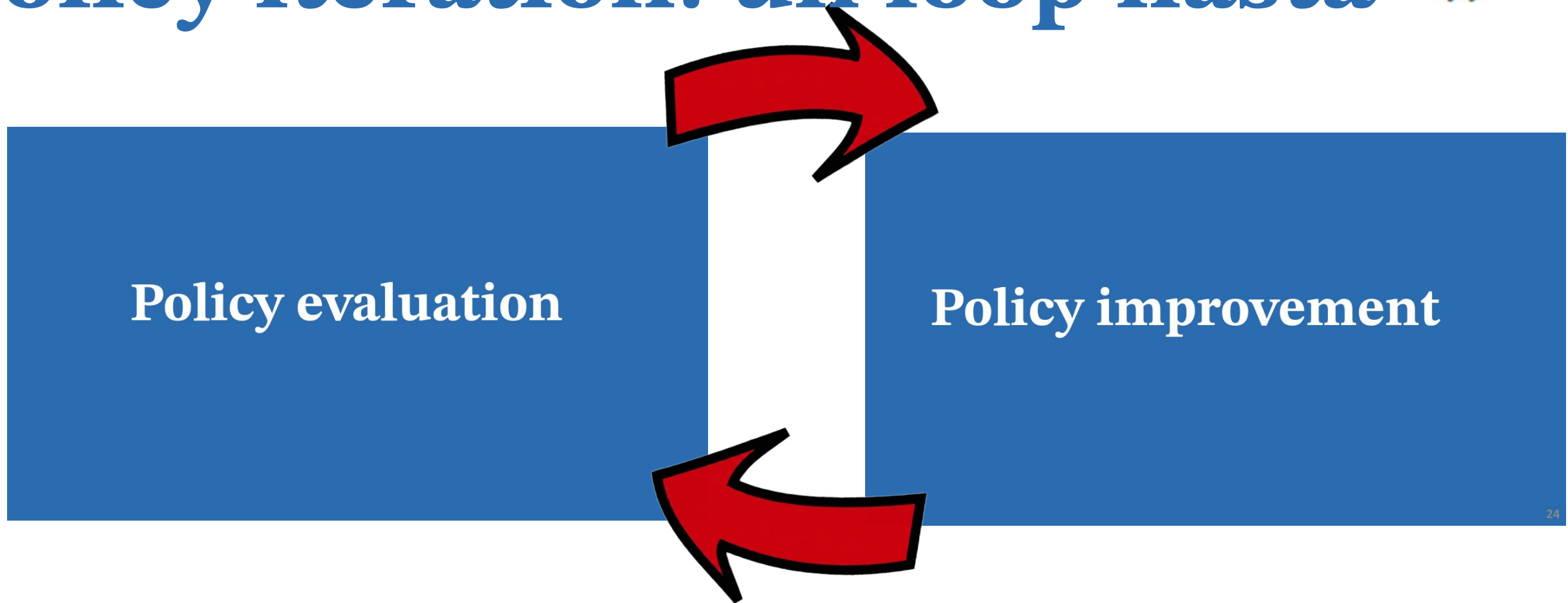
$old-action \leftarrow \pi(s)$

$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$



Policy iteration

Policy iteration: un loop hasta π^*



Si pensamos en MDPs finitos, entonces eventualmente vamos a encontrar el óptimo con esta idea!

Policy iteration: un loop hasta π^*

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$; $V(\text{terminal}) \doteq 0$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

Policy evaluation

3. Policy Improvement

policy-stable $\leftarrow$ true

For each $s \in \mathcal{S}$:

old-action $\leftarrow \pi(s)$

$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ false

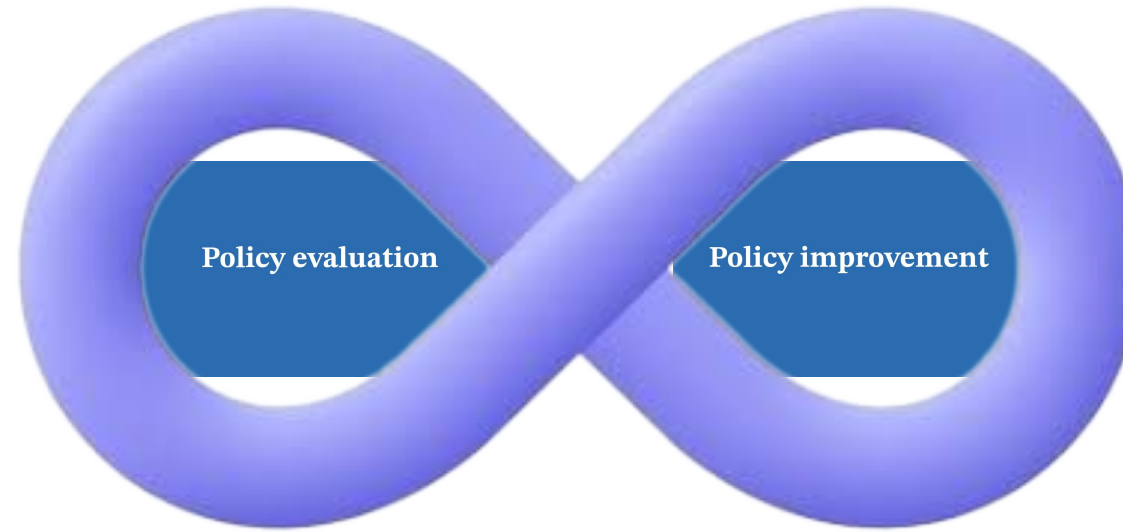
If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Policy improvement



Value iteration

Value iteration



*Es policy prediction sin converger (un sweep de $\mathcal{S}$),
Es Policy improvement (aplicar Optimal Bellman op.)*

$$v_{k+1}(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_k(s')]$$



Value iteration

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

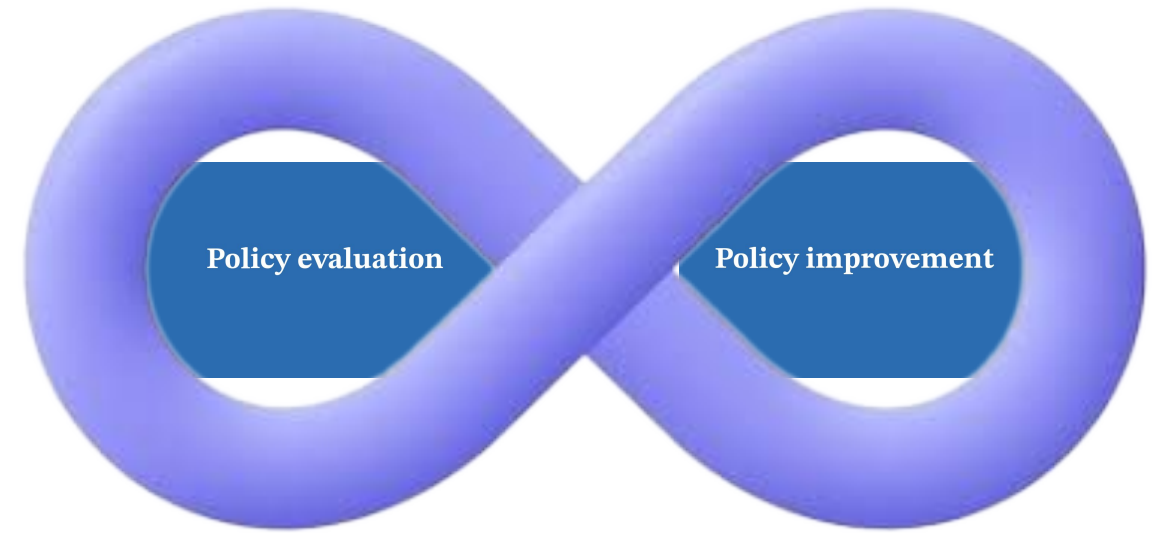
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

```
|  $\Delta \leftarrow 0$   
| Loop for each  $s \in \mathcal{S}$ :  
|    $v \leftarrow V(s)$   
|    $V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$   
|    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$   
until  $\Delta < \theta$ 
```

Output a deterministic policy, $\pi \approx \pi_*$, such that

$\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$



Es policy prediction sin converger (un sweep de $\mathcal{S}$),
Es Policy improvement (aplicar Optimal Bellman op.)

$$v_{k+1}(s) = \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma v_k(s')]$$



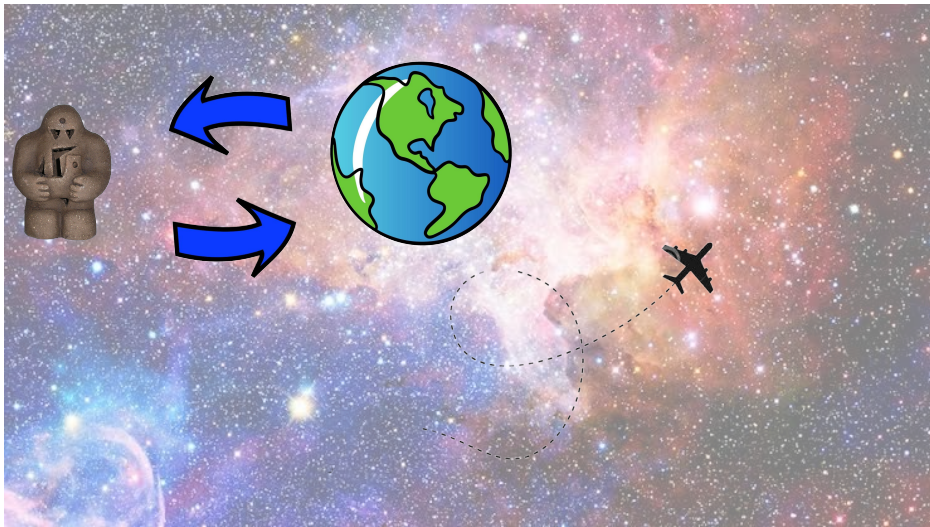


Asynchronous DP

Todo esto es muy “flexible”

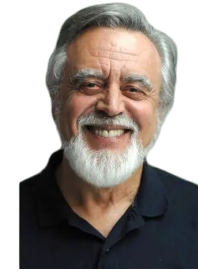
Value-iteration → hacer un single-step de predicción
→ \equiv single-step Opt. Bell

Asynchronous DP → diferentes combinaciones posibles



¿Cómo recorro el espacio $\mathcal{S}$
para estimar los $v(s)$?
framework muy flexible!

Todo esto es muy “flexible”

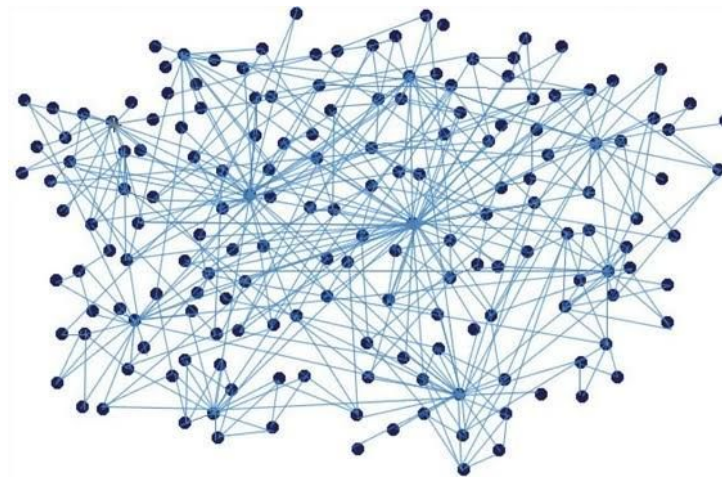
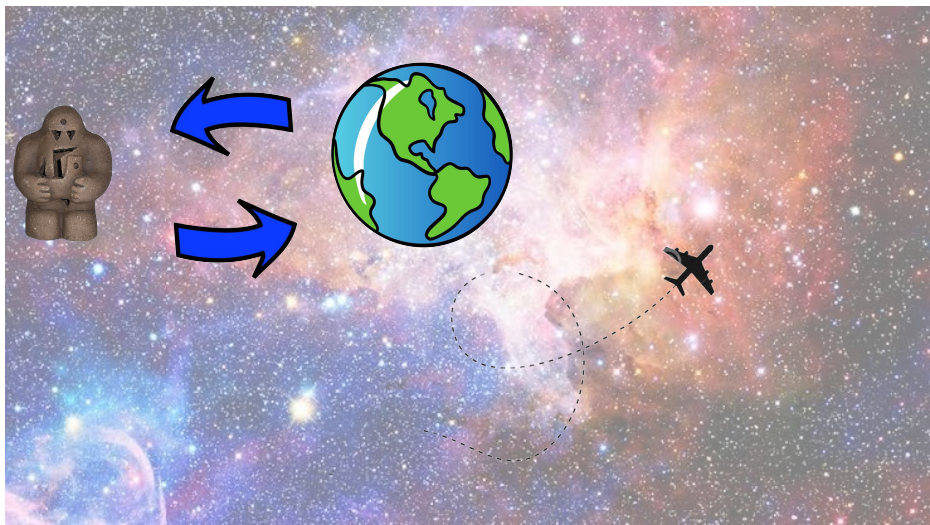


Bertsekas



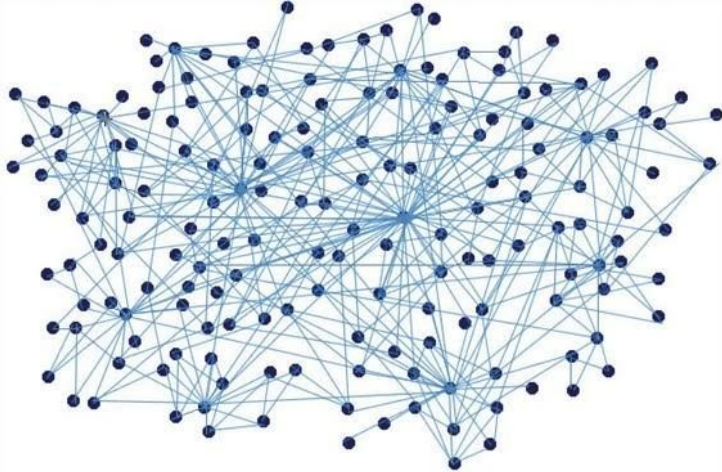
En esto de elegir qué estados “backupear”
hay un vínculo muy estrecho con shortest-path problems

$$d(s) = \min_{s': (s, s') \in E} [w(s, s') + d(s')], \quad d(\text{terminal}) = 0$$

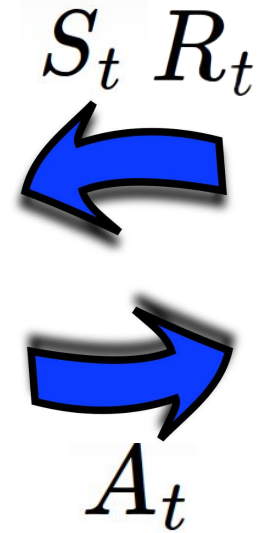


[\[Learning to act using real-time dynamic programming\]](#)

No perdamos el foco...



Agente



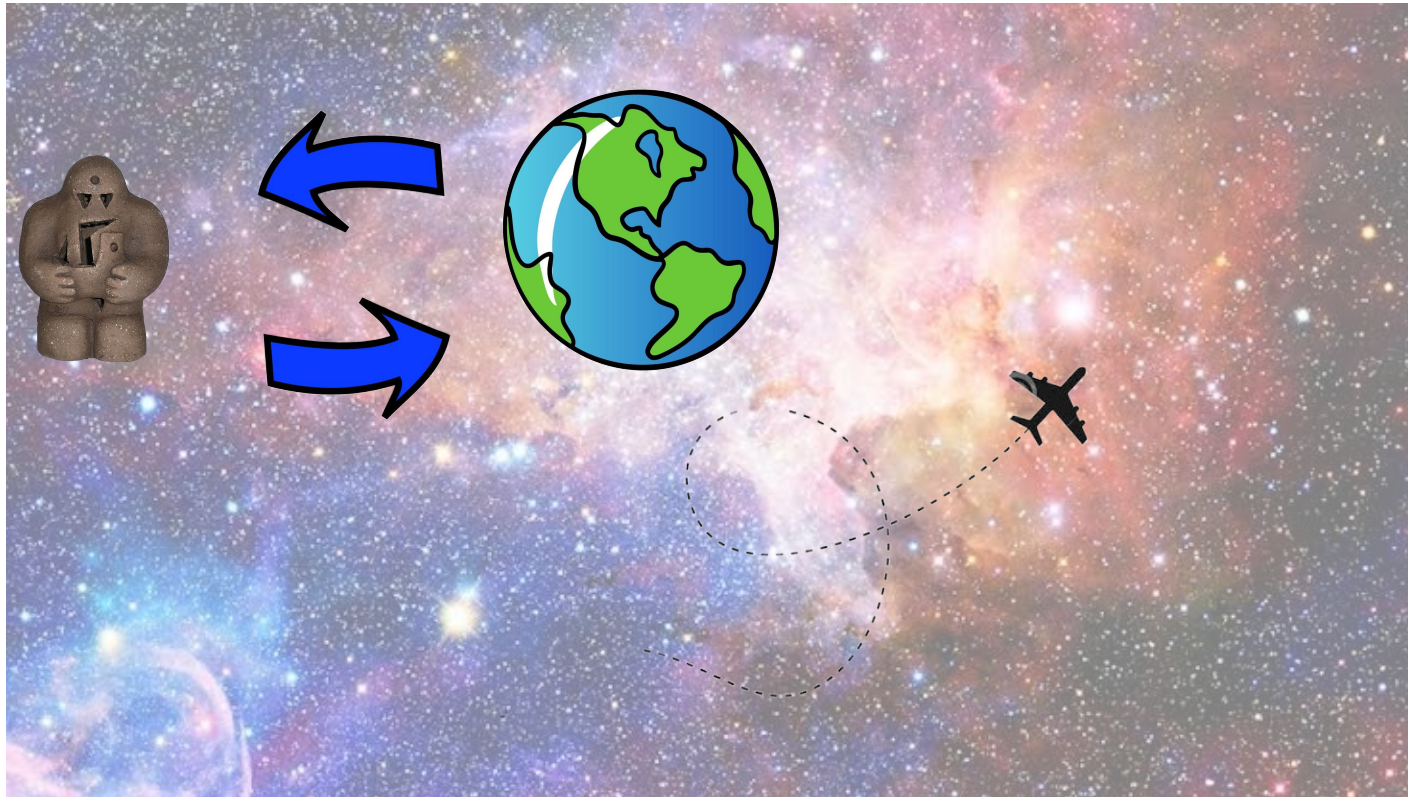
Entorno

Generalized policy iteration

Un marco para los algs de RL

GPI es la idea general de tener dos procesos que interactúan

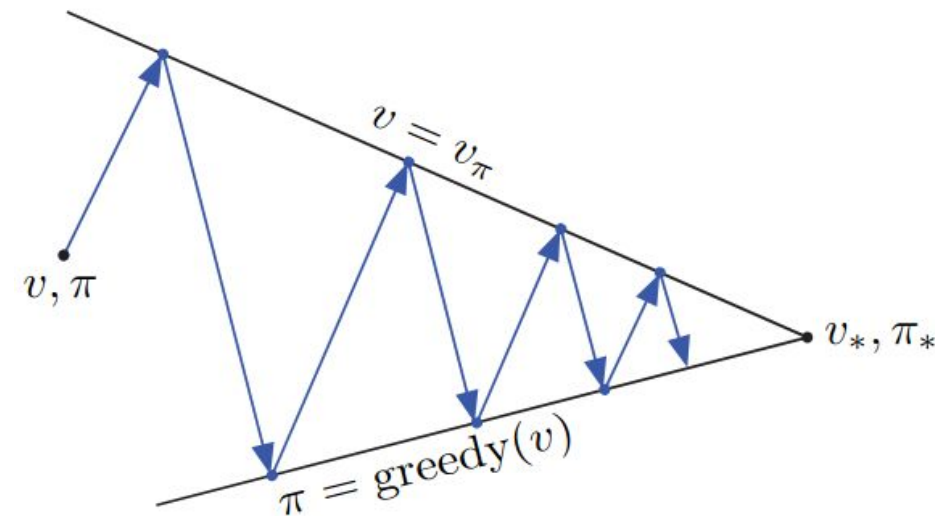
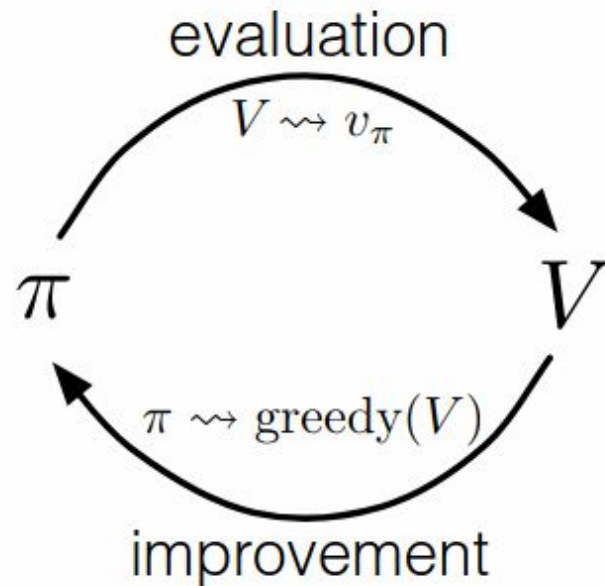
value function $\leftarrow$ GPI $\rightarrow$ *policy*



Un marco para los algs de RL

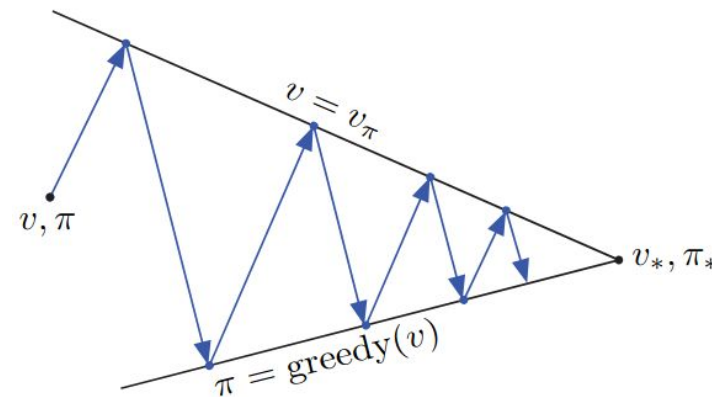
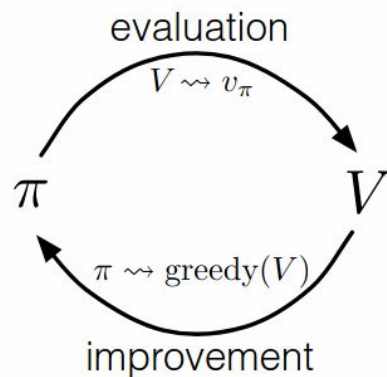
Evaluation $\rightarrow$ estimar $v(s)$ ||| Improvement $\rightarrow$ mejorar π

“compiten” y “cooperan”



Un marco para los algs de RL

Almost all reinforcement learning methods are well described as GPI. That is, all have identifiable policies and value functions, with the policy always being improved with respect to the value function and the value function always being driven toward the value function for the [current] policy", S&B



Memento práctica

Programamos un laberinto numpy

Policy prediction, improvement, iteration.

Value iteration.

Alguna idea de Asynchronous DP & Djistrka?

Gym

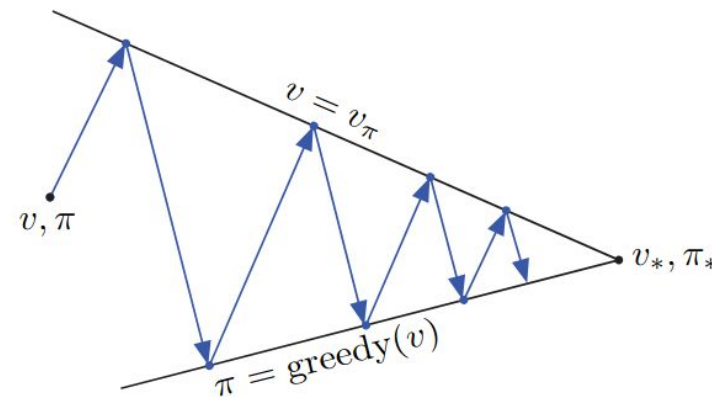
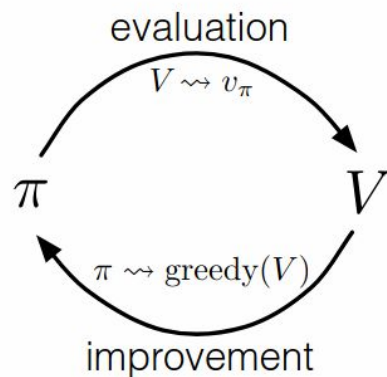
API: overview

FrozenLake

Taxi

Un marco para los algs de RL

Almost all reinforcement learning methods are well described as GPI. That is, all have identifiable policies and value functions, with the policy always being improved with respect to the value function and the value function always being driven toward the value function for the [current] policy", S&B

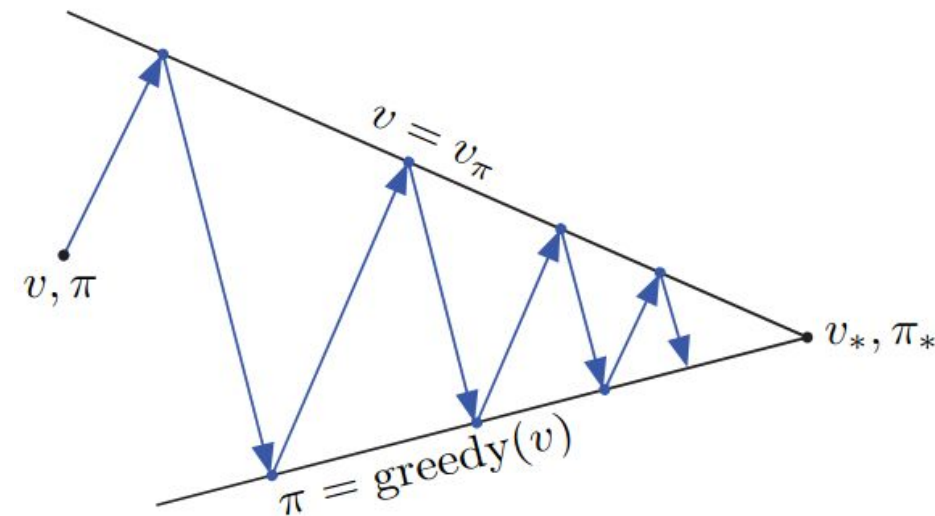
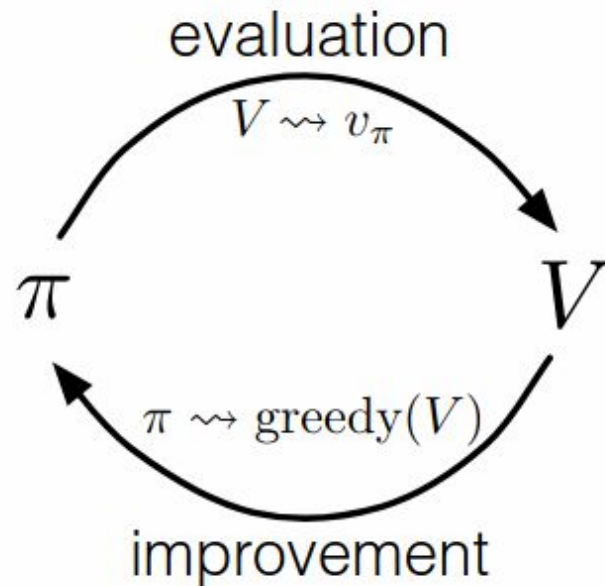


Gym

Un marco para los algs de RL

Evaluation $\rightarrow$ estimar $v(s)$ ||| Improvement $\rightarrow$ mejorar π

“compiten” y “cooperan”



Tarea

Tarea

Leer S&B Capítulo 4 (DP)

- * Mapear el laberinto de numpy a un [custom] env de Gym
 - * *Chusmear conexiones entre DP y shortest-path algs*
- (A, Djistrkra) [\[Learning to act using real-time dynamic programming\]](#)

(Procedimiento, lo mismo que las clases pasada!)